The Kernel Trick in Code

Contents

1. Overview 2
2. The data: two circles 4
3. A linear SVM fails 4
4. Lifting the data into 3D by hand 5
    4.1 The transformation 5
    4.2 Where to centre the bump 6
5. The RBF kernel 7
6. The polynomial kernel 7
7. Why it is a trick 9
8. Gamma: how far one point's influence reaches 10
9. An interactive playground 13
10. Summary 15
11. Sources 15
12. Key terms 16

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Kernel trick.
- **Compare with:** Polynomial features.

1. Overview

Key point: On data shaped as two circles, a linear SVM scores 55%. Switching the **kernel** (G-1004) to RBF, or to a degree-2 polynomial, scores 100%, using the same two input features.

The kernel trick showed in pictures how a kernel lifts data into a higher dimension. This Note runs it in scikit-learn on a real non-linear dataset, then looks at why no new features are needed. Figure 1 shows the four models we train. The ringed points are the **support vectors** (G-1923), the training points that fix the boundary; section 6 explains them.

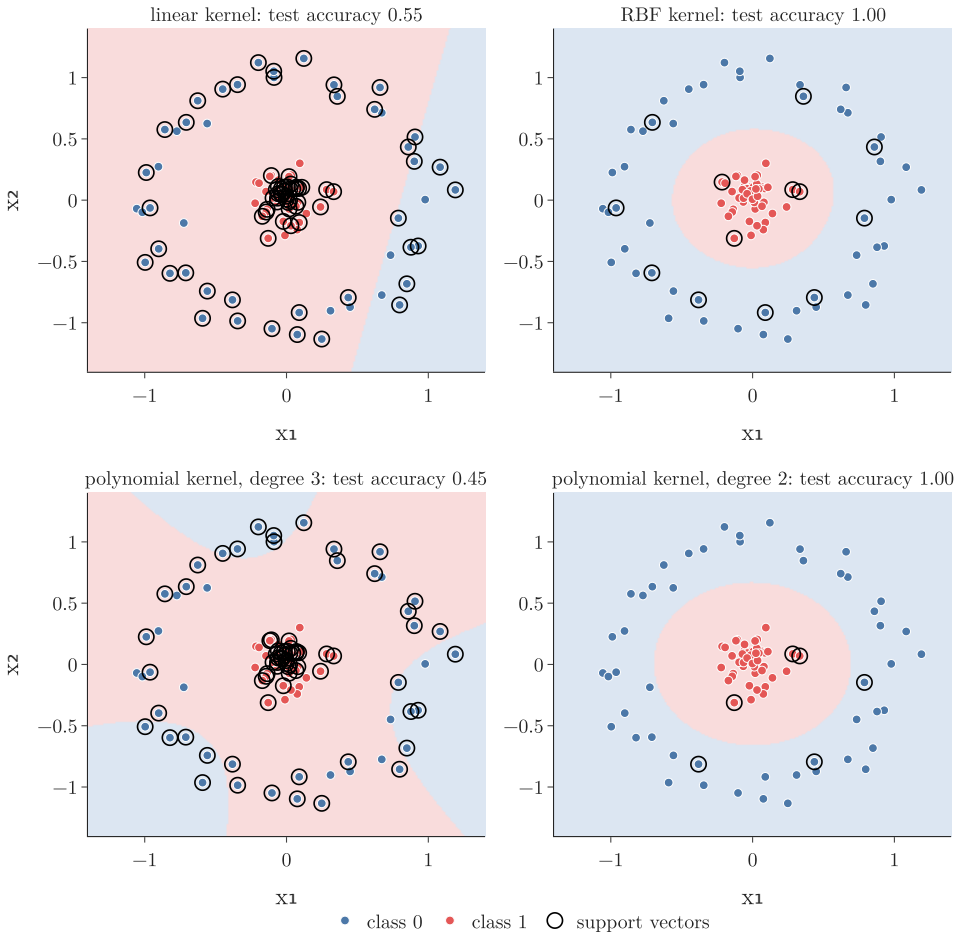


Figure 1: Decision regions of four SVMs on the circles data; ringed points are support vectors

This Note covers:

- the data: two circles (section 2);
- a linear SVM failing on it (section 3);
- lifting the data into 3D by hand (section 4);
- the RBF kernel (section 5) and the **polynomial kernel** (G-1514) (section 6);
- why the kernel is a trick: dot products without new features (section 7);
- gamma, the **RBF kernel** (G-1639)'s main setting (section 8);
- an interactive playground (section 9).

All the code is in the Notebook of this Note (`ML-090-kernel-trick-code.ipynb`).

2. The data: two circles

> **Key point:** make_circles generates 100 points: one class in a small inner circle, the other in a ring around it.

scikit-learn's sample generators create toy datasets. `make_circles` makes two concentric circles of points: 100 **observations** (one record each, one row of the data table), 50 per class, with two **features** (input variables, one column each): the x and y coordinates. The **target** (the output we predict) is the class. Class 1 (red) sits in the centre and class 0 (blue) in a ring around it.

> **Python:** Generating the data.

```python
from sklearn.datasets import make_circles

# factor: inner radius as a share of the outer one
X, y = make_circles(n_samples=100, factor=0.1,
                    noise=0.1, random_state=5)
X.shape        # (100, 2): x and y coordinates
```

Wherever we draw a straight line through this data, it cuts the ring in two, so no line can be a good classifier.

3. A linear SVM fails

> **Key point:** SVC(**kernel** (G-1004)="linear") reaches only 55% test accuracy: a straight decision boundary cannot separate a disc from a ring.

We split the data into training and test sets, train an SVM with a linear kernel, and measure accuracy on the test set.

> **Python:** Training a linear SVM.

```python
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0)

linear = SVC(kernel="linear")
linear.fit(X_train, y_train)
accuracy_score(y_test, linear.predict(X_test))   # 0.55
```

The accuracy is 0.55, little better than guessing, which was bound to happen with a linear classifier on non-linear data. To see the **decision boundary** (G-555), the line or curve where the model's prediction switches from one class to the other, a helper function predicts a fine grid of points and colours each by its prediction: these are the model's decision regions (see how the model predicts). Figure 1, top left, shows the result: one straight cut that leaves

many blue ring points on the red side.

4. Lifting the data into 3D by hand

> **Key point:** Put each coordinate through $\exp(-x^2)$ and add the results as a height z. The centre points rise to about 2, the ring stays below about 1.6, and a flat plane separates them.

4.1 The transformation

> **Key point:** $z = e^{-x_1^2} + e^{-x_2^2}$: large near the centre, small far from it.

A 2D example: concentric circles lifts these circles with $z = e^{-(x_1^2+x_2^2)}$, so the centre rises and a flat plane splits the classes. In code we use a per-coordinate variant: the bump e^{-x^2} applied to each coordinate, then added.

$$z = e^{-x_1^2} + e^{-x_2^2}$$

The per-coordinate bump behaves the same way: points near the centre, where both coordinates are small, get the largest z.

- A centre point, $(0.1, 0.05)$:

$$z = e^{-0.01} + e^{-0.0025}$$

$$z = 0.990 + 0.998 = 1.99$$

- A ring point, $(1, 0)$:

$$z = e^{-1} + e^{0}$$

$$z = 0.368 + 1 = 1.37$$

On the whole dataset, the centre points get z between 1.89 and 2.00, and the ring points between 1.01 and 1.56. So a flat plane at $z = 1.75$ separates them, and a linear classifier now works. Figure 2 animates the lift: watch the red centre points rise above the plane while every blue ring point stays below it.

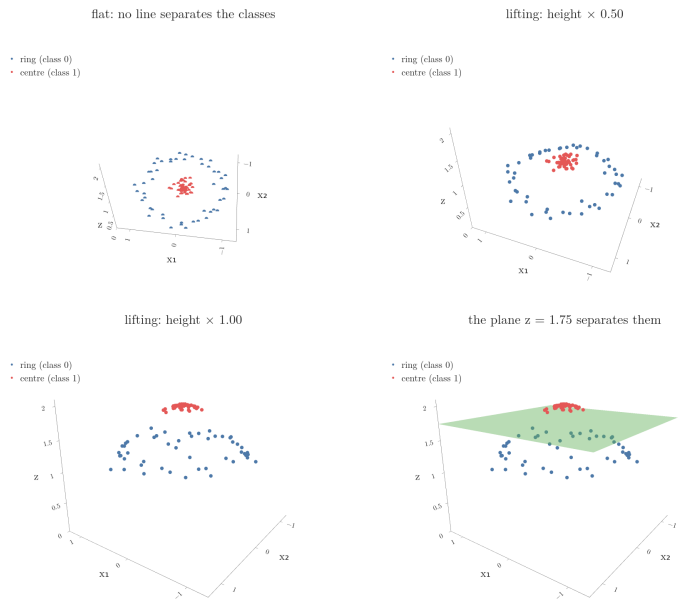


Figure 2: The circles data lifted by $z = e^{-x_1^2} + e^{-x_2^2}$: the centre class rises to 1.89–2.00, the ring stays at 1.01–1.56, and the plane $z = 1.75$ separates them

Python: The transformation and the 3D plot.

```
import numpy as np
import plotly.graph_objects as go

X_new = np.exp(-(X ** 2))      # exp(-x^2) for each coordinate
z = X_new.sum(axis=1)          # add the two columns: the height
go.Figure(go.Scatter3d(x=X[:, 0], y=X[:, 1], z=z,
                       mode="markers")).show()
```

4.2 Where to centre the bump

Key point: The bump only works if it is centred on the right class. Without knowing the centre, we would need one bump per point, which is expensive.

The lift worked because the bump e^{-x^2} is centred at 0, exactly where the inner class sits. If the inner class were centred at 5, we would need $e^{-(x-5)^2}$ instead. In higher dimensions we cannot look at the data and see where the centre is.

So SVM in effect places a bump on the data points themselves and adds the bumps up, each with its own weight; training finds the weights that separate the classes best. Doing this transformation explicitly would be very expensive as the data grows. The point of the **kernel trick** (G-1008) is that SVM never has to build these new features at all (Section 7).

Extra: The function used here, $e^{-x_1^2} + e^{-x_2^2}$, treats each coordinate separately; it is a convenient choice for the picture. The **RBF kernel** (G-1639) that SVM uses compares

two points through their distance: $K(a, b) = e^{-\gamma\|a-b\|^2}$. The SVM's decision function is a weighted sum of such bumps, one centred on each **support vector** (G-1923): $\sum_{i \in SV} y_i \alpha_i K(x_i, x) + b$ (sklearn UG §1.4.7). Here $\sum_{i \in SV}$ means "add over every support vector i", and the weights α_i are found in training.

5. The RBF kernel

Key point: SVC(**kernel** (G-1004)="rbf") reaches 100% test accuracy, with the same two features and no feature engineering.

Now we let SVM do it. We train a new SVC with the **RBF kernel** (G-1639), `kernel="rbf"`, instead of `"linear"` and change nothing else.

Python: The RBF kernel.

```
rbf = SVC(kernel="rbf")      # "rbf" is also the default
rbf.fit(X_train, y_train)
accuracy_score(y_test, rbf.predict(X_test))    # 1.0
```

The test accuracy is 1.0. Figure 1, top right, shows its **decision boundary** (G-555): a closed curve around the centre class.

We did not create any extra feature: only the original two features went into the model, and the kernel did the calculation internally. Compare this with **polynomial regression** (G-1515), where we had to add a new feature for every power of the inputs.

6. The polynomial kernel

Key point: kernel (G-1004)="poly" uses degree 3 by default and scores 45% here. With degree=2 it scores 100%. The degree is a hyperparameter.

The **polynomial kernel** (G-1514) is chosen with `kernel="poly"`. The polynomial kernel has an extra setting, `degree`, the degree of the polynomial. The default degree is 3, and it is ignored by all other kernels.

Kernel	Test accuracy	Support vectors (of 80)
linear	0.55	76
RBF	1.00	13
polynomial, degree 3	0.45	74
polynomial, degree 2	1.00	6

With the default degree 3, the accuracy is even worse than linear: 0.45, with the odd-shaped **decision boundary** (G-555) of Figure 1, bottom left. With `degree=2` it jumps to 1.0 (bottom right). The degree must therefore be tuned like any **hyperparameter** (G-910), for example with a **grid search** (trying every combination of listed values) and **cross-validation** (scoring each setting on several held-out parts of the data; see cross-validation with a pipeline).

Python: Changing the degree.

```
for degree in [3, 2]:
    poly = SVC(kernel="poly", degree=degree)
    poly.fit(X_train, y_train)
    print(degree, accuracy_score(y_test,
                                 poly.predict(X_test)))
# 3 0.45
# 2 1.0
```

Extra: Why does degree 2 work and degree 3 fail? scikit-learn's polynomial kernel is $(\gamma\, a \cdot b + r)^d$ with r (`coef0`) equal to 0 by default (sklearn UG §1.4.6). With $r = 0$, the degree-3 kernel contains only terms of degree exactly 3, such as x_1^3 and $x_1^2 x_2$. A circle needs $x_1^2 + x_2^2$, a degree-2 term, which the degree-3 kernel cannot produce. The degree-2 kernel contains exactly x_1^2, $x_1 x_2$ and x_2^2. Setting `coef0=1` with degree 3 adds the lower-degree terms back, and in the Notebook this model also scores 1.00 on the circles.

Figure 3 shows why the degree-2 terms are enough. Watch the circle on the left turn into a straight line on the right once each coordinate is squared: in the features x_1^2 and x_2^2 the decision boundary $x_1^2 + x_2^2 = 0.34$ is linear.

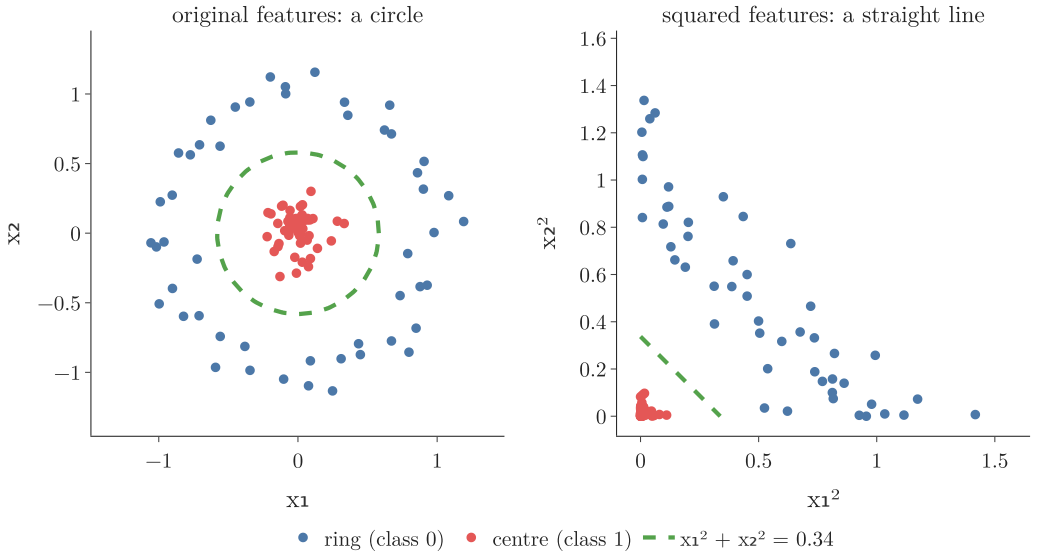


Figure 3: Left: the circles data and the circle $x_1^2 + x_2^2 = 0.34$ between the classes. Right: the same points in the squared features, where that circle is the straight line $x_1^2 + x_2^2 = 0.34$

The table also shows the **support vectors** (G-1923). The linear model needs almost every training point; the good kernels need only a few. The reason: every point that sits on the margin, inside it or on the wrong side of the boundary becomes a support vector (ESL §12.2.1). A straight line fits the circles badly, so most points end up inside its margin; a curved boundary that fits well leaves only a few points near it.

Extra: The Notebook checks the margin rule for the linear model. The linear model misclassifies 30 of the 80 training points, and exactly 76 points have $y \cdot f(x) < 1$ (inside the margin or on the wrong side): the same 76 as its support vectors.

7. Why it is a trick

Key point: A **kernel** (G-1004) gives the dot product of two points in the higher-dimensional space directly from the original coordinates, so the new features are never built.

SVM's training and predictions only need **dot products** (G-634; multiply matching entries of two vectors and add, see computing the dot product) between pairs of points (MML §12.4). A kernel is a function $K(a, b)$ that returns the dot product the two points would have **after** the transformation, without carrying the transformation out. We put the original coordinates into the kernel formula, and out comes the value we need.

As a formula, for the degree-2 **polynomial kernel** (G-1514) the explicit features are:

$$\phi(x) = (x_1^2,\ \sqrt{2}\, x_1 x_2,\ x_2^2)$$

The kernel is:

$$K(a, b) = (a \cdot b)^2 = \phi(a) \cdot \phi(b)$$

With numbers, for $a = (1, 2)$ and $b = (3, 1)$:

- Kernel:

$$a \cdot b = 3 + 2 = 5$$

$$K(a, b) = 5^2 = 25$$

- Explicit features:

$$\phi(a) = (1,\ 2\sqrt{2},\ 4)$$

$$\phi(b) = (9,\ 3\sqrt{2},\ 1)$$

$$\phi(a) \cdot \phi(b) = 9 + 12 + 4 = 25$$

Figure 4 draws the two routes side by side; watch both arrive at 25, while only the top route builds new features.

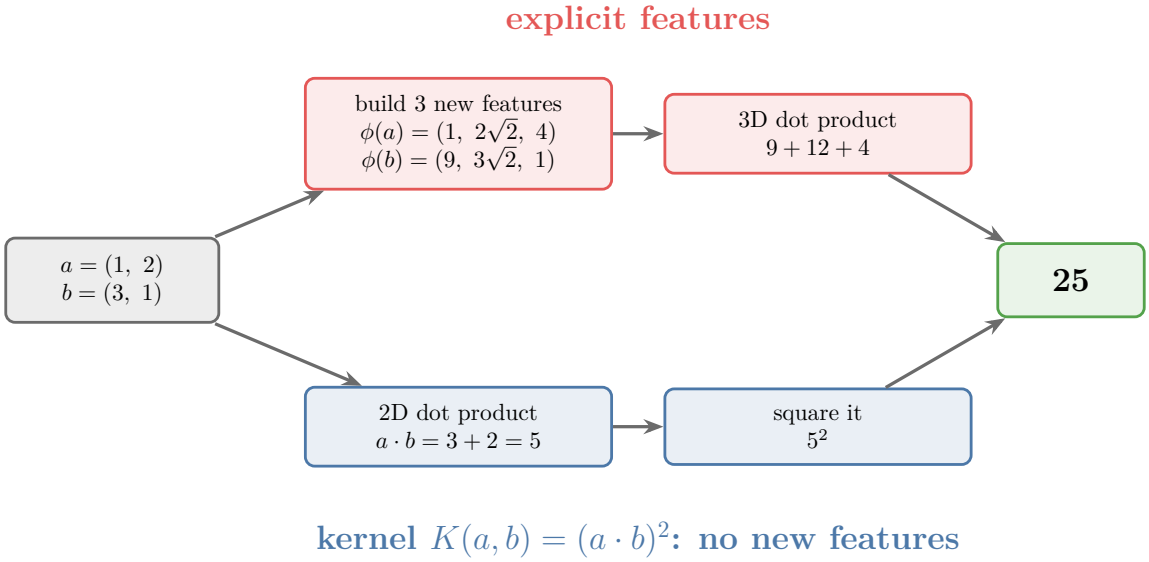


Figure 4: Two routes to $\phi(a) \cdot \phi(b)$ for $a = (1, 2)$ and $b = (3, 1)$. Top: build the three degree-2 features, then take a 3D dot product. Bottom: the kernel, a 2D dot product squared

The kernel needed one 2D dot product and a square; the explicit route needed three new features per point. With more inputs and higher degrees the explicit features multiply quickly, while the kernel stays one dot product. For the **RBF kernel** (G-1639) the matching feature space is infinite-dimensional, so building it would be impossible, yet the kernel value is cheap (MML §12.4). Getting the dot product without building the features is exactly the **kernel trick** (G-1008).

8. Gamma: how far one point's influence reaches

> **Key point:** The **RBF kernel** (G-1639) value between two points falls as their distance grows. Gamma sets how fast. Small gamma: far points still count, smooth boundary. Large gamma: only very close points count, and the boundary wraps around single points.

The RBF kernel compares two points through their distance:

$$K(a, b) = e^{-\gamma \|a-b\|^2}$$

Here $\|a - b\|$ is the distance between points a and b. For $a = (1, 2)$ and $b = (4, 6)$:

$$a - b = (-3, -4)$$

$$\|a - b\| = \sqrt{(-3)^2 + (-4)^2}$$

$$\|a - b\| = \sqrt{9 + 16} = 5$$

With $\gamma = 0.1$, the **kernel** (G-1004) value of these two points is:

$$K(a, b) = e^{-0.1 \times 5^2} = e^{-2.5}$$

$$K(a, b) = 0.08$$

The value is 1 when the two points coincide and falls towards 0 as they move apart. We can read the value as the **influence** one training point has on another point: close points have a lot of influence, far points almost none. The setting **gamma** (G-823), γ, multiplies the squared distance, so gamma decides how quickly the influence dies out.

For two points at distance 1, $K = e^{-\gamma}$:

gamma (G-823)	K at distance 1	Reading
0.1	$e^{-0.1} = 0.90$	a point at distance 1 still has almost full influence
1	$e^{-1} = 0.37$	about a third
10	$e^{-10} = 0.00005$	practically none

Figure 5 draws the kernel value against the distance for the three gammas. Watch the red curve (gamma = 10): the influence is gone before distance 1, while the green curve (gamma = 0.1) is still about 0.4 at distance 3.

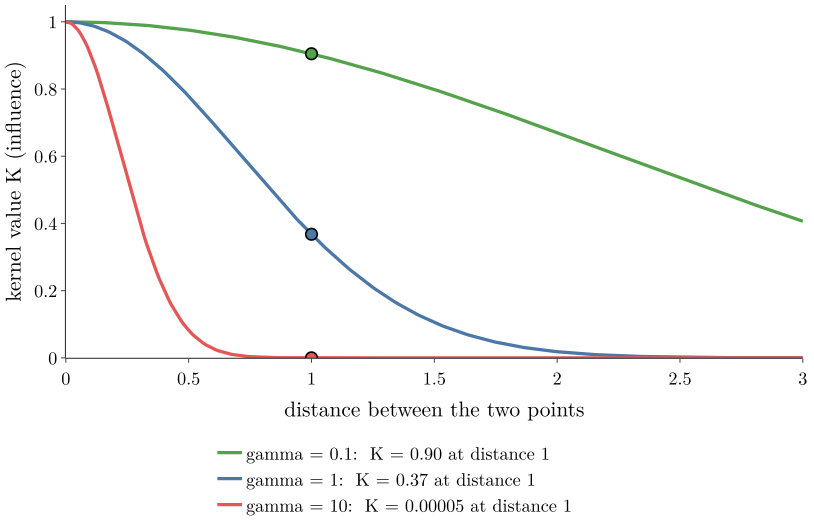


Figure 5: The RBF kernel value against the distance between two points, for gamma = 0.1, 1 and 10. The dots mark distance 1: 0.90, 0.37 and 0.00005

A small gamma makes each bump wide, so many training points have a say at every location and the **decision boundary** (G-555) is smooth (risk of underfitting). A large gamma makes each bump narrow, so only the nearest training points have a say and the boundary wraps around individual points (risk of overfitting). Gamma is a **hyperparameter** (G-910).

Figure 6 runs a gamma sweep on the moons data (two interleaved half-circles of points), from 0.01 to 1000, with C = 1 (the setting that trades a wide margin against training mistakes, see the hyperparameter C). Watch the decision boundary go from an almost straight cut to a curve that follows the moons, then break into small islands around single points. Training accuracy climbs to 1.00, while accuracy on 5,000 fresh points from the same generator peaks at 0.97 (gamma about 5) and falls to 0.68 at gamma 1000.

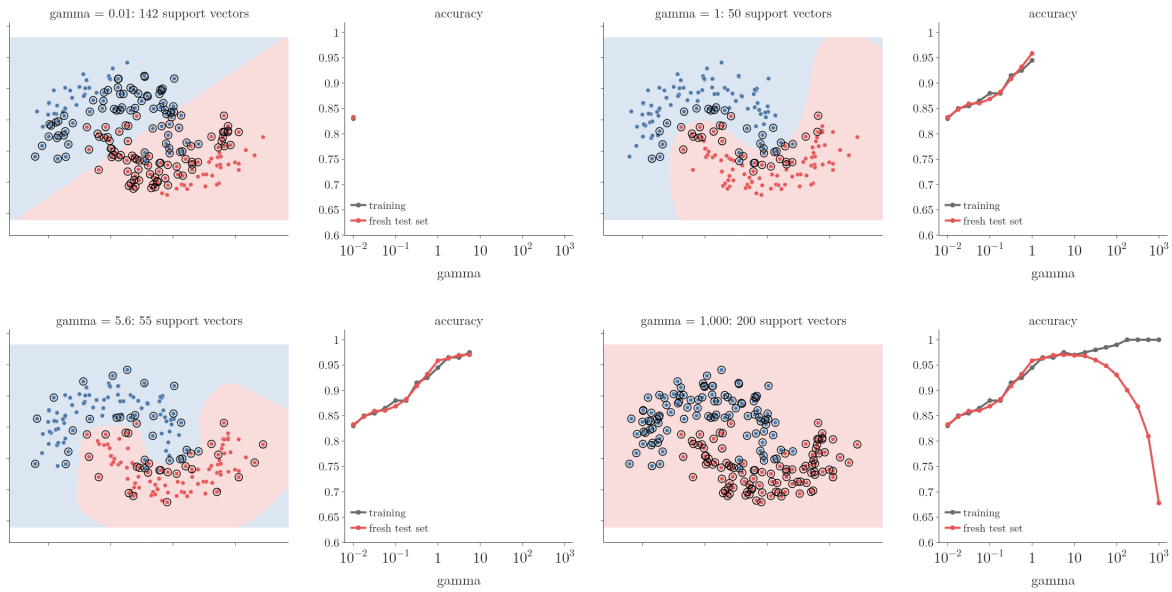


Figure 6: RBF SVM on the moons data as gamma grows. Left: decision regions, with support vectors ringed. Right: training accuracy and accuracy on a fresh test set of 5,000 points

> **Extra:** scikit-learn's default, `gamma="scale"`, sets gamma from the spread of the data: $1/(p \times \text{variance of } X)$, where p is `n_features`, the number of features. C and gamma are usually tuned together with a grid search (sklearn, "RBF SVM parameters").

9. An interactive playground

> **Key point:** app.py lets you change the dataset, **kernel** (G-1004), C, gamma and degree and watch the decision regions and support vectors update.

The folder of this Note contains `app.py`, a small Dash app. Run `python app.py` and open `http://127.0.0.1:8050`. Figure 7 shows it with its default settings.

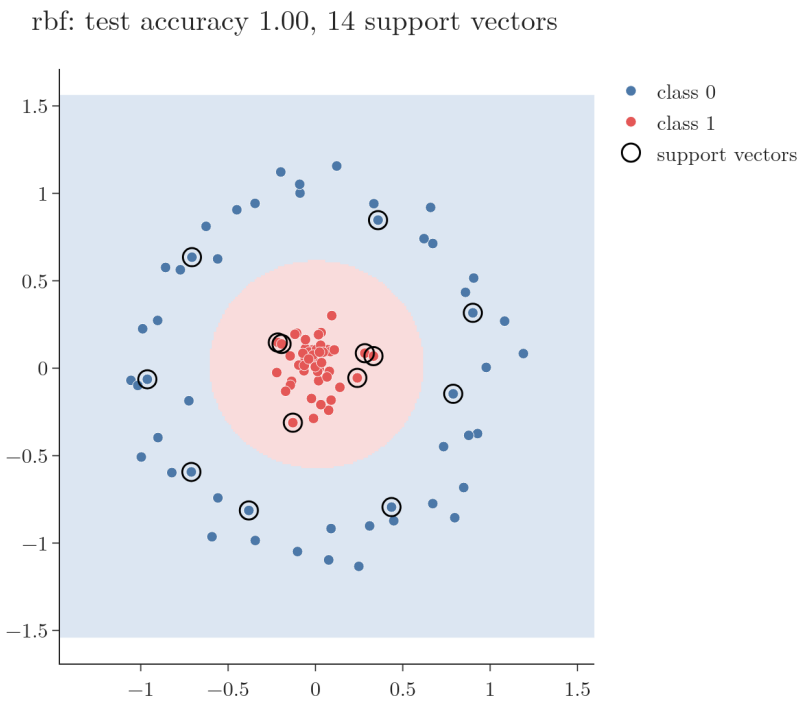


Figure 7: The playground on the circles data with the RBF kernel, C = 1 and gamma = 1

Things to try:

- **linear** on the circles: one straight cut, about 0.55 accuracy, and nearly every point a **support vector** (G-1923).
- **poly** with degree 3, then 2: the accuracy jumps from 0.45 to 1.00.
- **C** from 3 down to −3 (that is, 10^3 to 10^{-3}): small C widens the margin and adds support vectors, as in the hyperparameter C.
- **gamma** with the **RBF kernel** (G-1639) on the moons data: from smooth boundaries at small gamma to tight islands around single points at large gamma, as in Figure 6.

10. Summary

Model	Code	Test accuracy
Linear SVM	`SVC(kernel="linear")`	0.55
RBF kernel	`SVC(kernel="rbf")`	1.00
Polynomial, degree 3 (default)	`SVC(kernel="poly")`	0.45
Polynomial, degree 2	`SVC(kernel="poly", degree=2)`	1.00

- A linear SVM cannot separate circular classes, because any straight line cuts the ring in two.
- Lifting with $z = e^{-x_1^2} + e^{-x_2^2}$ makes them separable by a plane in 3D, because the centre points rise (1.89 to 2.00) while the ring stays low (1.01 to 1.56).
- With a **kernel** (G-1004), SVM gets the same effect from the original two features: no new features built, so there is no need to know where to centre the bump or to pay for building the features.
- The kernel's settings (degree, **gamma** (G-823)) and C are hyperparameters to tune, because the wrong setting fails: degree 3 scores 0.45 on the circles where degree 2 scores 1.00.
- Gamma sets how far one point's influence reaches: small gamma gives smooth boundaries, large gamma tight ones; so too large a gamma overfits (moons test accuracy falls from 0.97 to 0.68).
- A kernel returns the dot product in the higher-dimensional space directly: the **kernel trick** (G-1008). It works because SVM needs only dot products, so one formula on the original coordinates replaces many new features.
- So in code, switching the kernel is a one-word change that turns a 55% model into a 100% model on data no straight line can split.

11. Sources

Built from

- CampusX, "Kernel Trick in SVM | Code Example", YouTube, https://www.youtube.com/watch?v=pjv
- StatQuest with Josh Starmer, "Support Vector Machines Part 3: The Radial (RBF) Kernel (Part 3 of 3)", YouTube, https://www.youtube.com/watch?v=Qc5IyLW_hns (gamma scales the squared distance, and so the influence two points have on each other; section 8)

Other references

- **ESL:** T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer, 2009. Section 12.2.1.
- **MML:** M. P. Deisenroth, A. A. Faisal and C. S. Ong, *Mathematics for Machine Learning*, Cambridge University Press, 2020. Section 12.4, Kernels.
- **sklearn UG:** scikit-learn User Guide, Section 1.4, Support Vector Machines (1.4.6 Kernel functions, 1.4.7 Mathematical formulation), and the `SVC` reference page. scikit-learn.org/stable/modules/svm.html

- **sklearn, "RBF SVM parameters":** scikit-learn example gallery, *RBF SVM parameters*. scikit-learn.org, auto_examples/svm/plot_rbf_parameters

12. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
gamma (RBF kernel) (G-823)	How far one point's influence reaches in the RBF kernel; large gamma gives tighter boundaries.
Kernel function $K(a, b)$ (G-1006)	A function of two points that returns the dot product they would have after lifting to a higher-dimensional space, $\phi(a) \cdot \phi(b)$, computed straight from the original points, so the lifted features are never built.
Kernel (SVM) (G-1004)	A function of two points, $K(a, b)$, that returns their dot product in the higher-dimensional space, $\phi(a) \cdot \phi(b)$, directly from the original values (not the Jupyter kernel).
Support vectors (G-1923)	The training points that lie on the two margin lines, π^+ or π^-; they alone fix the SVM line.
Polynomial kernel (G-1514)	A function that gives the dot product of two points after their inputs are expanded into powers, such as x^2 (a kernel whose feature map is built from powers of the inputs); it lets an SVM draw curved boundaries.
RBF kernel (G-1639)	Radial basis function kernel, built on $e^{-(\text{distance})^2}$: it scores two points as similar (close to 1) when they are near each other and close to 0 when far apart; the usual first-choice SVM kernel for curved boundaries.
Decision boundary (G-555)	A line or curve that separates the classes in classification.
Kernel trick (G-1008)	Making non-linear data separable by mapping it to a higher dimension, without building the new columns.
Polynomial regression (G-1515)	Linear regression on powers (and products) of the inputs, to fit curves.

Term	Meaning
Hyperparameter (G-910)	A setting of an algorithm chosen before training, such as a tree's `max_depth`.
Dot product (G-634)	Multiplying two vectors entry by entry and adding the products, $u^\top x$, which gives one number. It is large when the vectors point the same way, so it measures similarity and gives projections.
`make_circles` (G-110)	scikit-learn function that generates two concentric rings of observations, one ring per class.
Feature map (ϕ) (G-765)	A rule that gives each point extra coordinates built from its own values, such as $\phi(x) = (x, x^2)$, moving the data into a higher-dimensional space. Data that no straight line can separate becomes linearly separable there.
Degree (G-577)	The highest power used in a polynomial, such as 2 for x^2. It sets the model's complexity: a higher degree lets the curve bend more, and too high a degree overfits.